

bok25

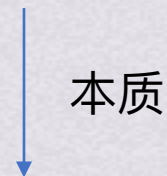
基
礎
班

、**考研数据結構**



遍历的本质

遍历可以以一定规则将二叉树的节点排列成一个线性序列

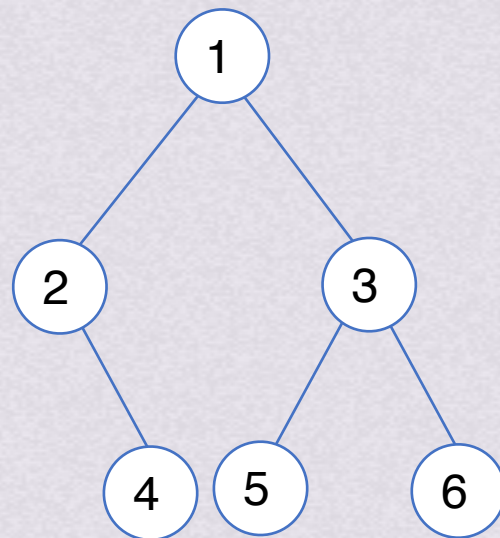


对一个非线性结构进行线性化操作，
使除第一个和最后一个节点外的所有节点
在这些线性序列中有且仅有一个直接前驱和直接后继



线索二叉树

对于这样的二叉树：



先序遍历输出序列: 1 2 4 3 5 6

中序遍历输出序列: 2 4 1 5 **3** 6

后序遍历输出序列: 4 2 5 6 3 1

是否可以直接看着树说出中序遍历下6的前驱？

常规做法：做一次中序遍历

线索二叉树：利用节点两个孩子的空位存储遍历顺序信息



线索二叉树

如果节点有左子树，lchild指向左孩子；否则指向其前驱

如果节点有右子树，rchild指向右孩子；否则指向其后继

lchild	ltag	data	rtag	rchild
--------	------	------	------	--------

ltag=0，lchild域指示节点的左孩子

ltag=1，lchild域指示的是前驱

rtag=0，rchild域指示节点的右孩子

rtag=1，rchild域指示的是后继

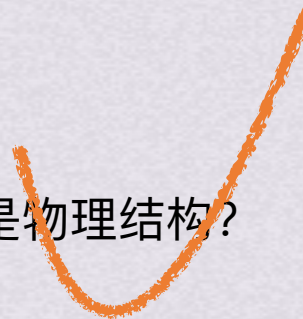


构造线索二叉树

以中序线索二叉树为例

线索化就是在遍历的过程中修改空指针

思考：线索二叉树是逻辑结构还是物理结构？



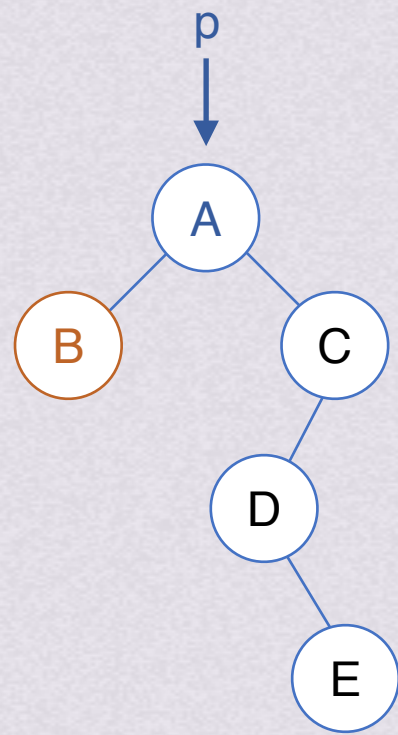
构造线索二叉树

以中序线索二叉树为例

【注】 初始pre=NULL

中序线索化以节点p为根的树

```
void InThreading(BiThrTree p)
{
    //pre 是全局变量，初始化时其右孩子指针为空，便于在树的最左点开始建线索
    if(p)
    {
        InThreading(p->lchild); //左子树递归线索化
        if(!p->lchild) //p的左孩子为空
        {
            p->LTag=1; //给p加上左线索
            p->lchild=pre; //p的左孩子指针指向pre（前驱）
        }
        else p->LTag=0;
        if(!pre->rchild) //pre的右孩子为空
        {
            pre->RTag=1; //给pre加上右线索
            pre->rchild=p; //pre的右孩子指针指向p（后继）
        }
        else pre->RTag=0;
        pre=p; //保持pre指向p
        InThreading(p->rchild); //右子树递归线索化
    }
}
```



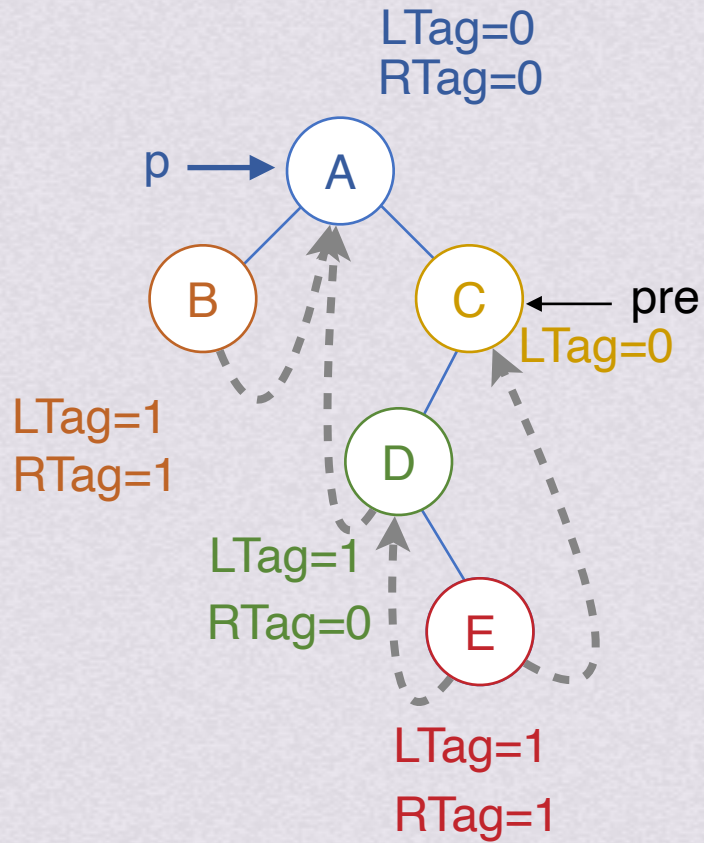
if(a)表示a为真则执行if循环
if(!a)表示a不为真则执行if循环

构造线索二叉树

以中序线索二叉树为例

中序线索化以节点p为根的树

```
void InThreading(BiThrTree p)
{
    //pre 是全局变量，初始化时其右孩子指针为空，便于在树的最左点开始建线索
    if(p)
    {
        InThreading(p->lchild); //左子树递归线索化
        if(!p->lchild) //p的左孩子为空
        {
            p->LTag=1; //给p加上左线索
            p->lchild=pre; //p的左孩子指针指向pre（前驱）
        }
        else p->LTag=0;
        if(!pre->rchild) //pre的右孩子为空
        {
            pre->RTag=1; //给pre加上右线索
            pre->rchild=p; //pre的右孩子指针指向p（后继）
        }
        else pre->RTag=0;
        pre=p; //保持pre指向p
        InThreading(p->rchild); //右子树递归线索化
    }
}
```



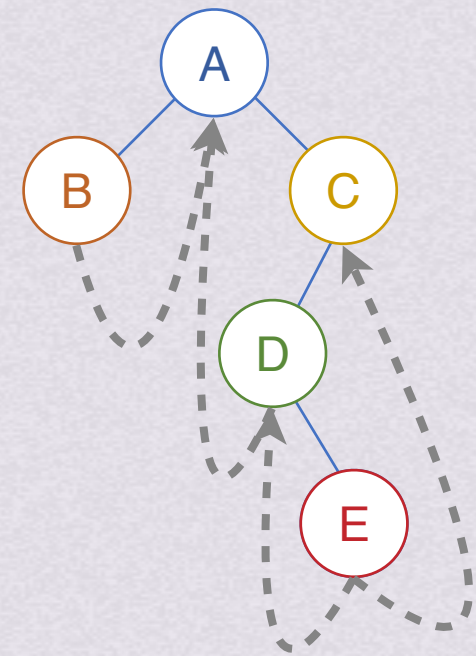
递归栈

构造线索二叉树

以中序线索二叉树为例

中序线索化以节点p为根的树

```
void InThreading(BiThrTree p)
{
    //pre 是全局变量，初始化时其右孩子指针为空，便于在树的最左点开始建线索
    if(p)
    {
        InThreading(p->lchild); //左子树递归线索化
        if(!p->lchild) //p的左孩子为空
        {
            p->LTag=1; //给p加上左线索
            p->lchild=pre; //p的左孩子指针指向pre (前驱)
        }
        else p->LTag=0;
        if(!pre->rchild) //pre的右孩子为空
        {
            pre->RTag=1; //给pre加上右线索
            pre->rchild=p; //pre的右孩子指针指向p (后继)
        }
        else pre->RTag=0;
        pre=p; //保持pre指向p
        InThreading(p->rchild); //右子树递归线索化
    }
}
```





构造线索二叉树

以中序线索二叉树为例

代码解构

- 访问p的左孩子
- 如果左孩子为空，对左孩子指针进行前驱线索化
- 如果前驱pre的右孩子为空，对前驱右孩子指针进行后继线索化
- pre追上p
- 访问p的右孩子